MINOR PROJECT ASSESSMENT-5

API & ANDROID SECURITY

Practical + Scenario-Based Technical Report

Submission Detail	Information
Student Name	THATHA VENKATA SURYA CHETHAN
Batch	Cybersecurity
Assessment	Minor Project Assessment: API & Android Security
Total Marks	100
Tools (as applicable)	Burp Suite Community, browser developer tools, crAPI, JADX, Android APK utilities

Document Control & Scope

Item	Details
Objective	Assess API attack surface, API security weaknesses, BOLA/access control, OTP workflow security, Android APK exposure, and professional pentester decision-making.
Authorized scope	Instructor-approved crAPI instance or equivalent lab; instructor-approved API endpoints; APK explicitly supplied/authorized for analysis; instructor-provided evidence files only.
Out of scope	Public or production targets without written authorization, real user data, credential abuse, destructive actions, bulk enumeration, and exploitation beyond the minimum proof needed.
Evidence standard	Screenshots and sanitized request/response excerpts must come from the student's own lab session. Mask tokens, cookies, personal data, and secrets.
Limitations	No live lab evidence is embedded in this document. Evidence boxes are intentionally left blank for authentic screenshots and observations.

Executive Summary

This assessment uses a controlled, authorization-first methodology to examine API and Android application security. The work covers endpoint discovery, API behavior testing, a multi-request crAPI investigation, object-level authorization, OTP verification controls, APK static analysis with JADX, and an integrated risk-based test plan.

The key security questions are whether the server verifies access to each requested object, validates all security-sensitive input, enforces OTP verification on the server, prevents replay and brute-force attempts, and keeps secrets out of client-side application packages.

Assessment outcome: Complete the status and finding summaries below after performing the authorized tests. Do not mark a weakness as confirmed based only on a suspicious endpoint, client-side code, or a hypothetical scenario.

Methodology

1. Confirm written authorization, target host, test accounts, permitted actions, and testing window.
2. Discover endpoints from the approved API specification, application traffic, and APK static analysis.
3. Establish normal baseline behavior using a valid test account and benign inputs.
4. Change one variable at a time; use only objects and accounts created for the lab wherever possible.
5. Capture sanitized request/response evidence and record expected versus actual behavior.
6. Stop when the minimum proof is sufficient; avoid collecting unrelated data or performing destructive actions.
7. Rate confirmed findings using likelihood, impact, and business context; provide practical remediation and retest criteria.

SECTION A — API SECURITY & crAPI

Q1. API Reconnaissance & Attack Surface Mapping (15 Marks)

Objective: Identify at least eight API endpoints in the authorized crAPI lab or equivalent. Use the OpenAPI document, application workflows, and Burp Suite HTTP history. The endpoint names below are examples of categories to look for, not a claim that each exact path exists in the target version. Verify the actual method and path in the lab before finalizing.

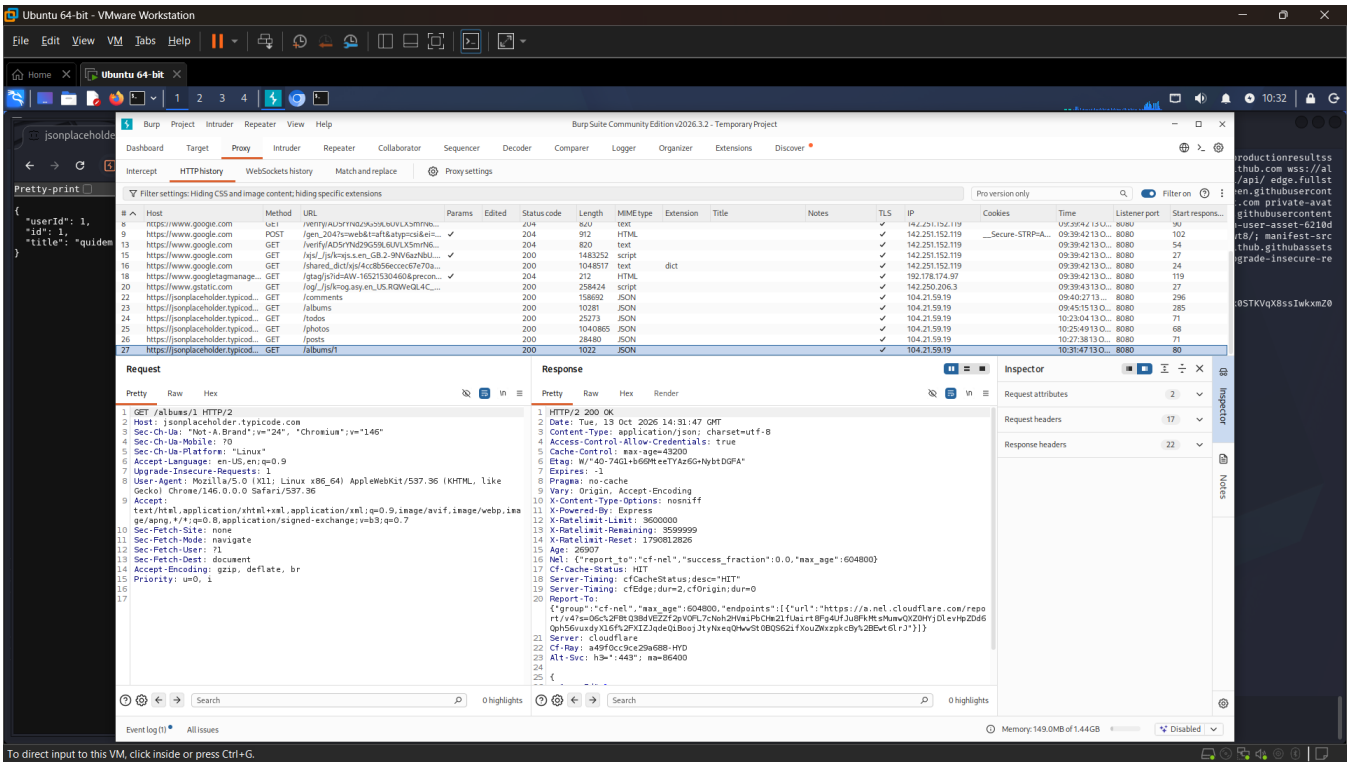
#	Method + Path (verify in lab)	Auth?	Important parameters	App/security function	Deep test?
1	[GET] /identity/.../login	No	username/email, password	Starts authentication	No
2	[POST] /identity/.../login	No	credentials; request body	Authenticates user and issues session/token	Yes — token/session handling
3	[POST] /identity/.../signup	No	email, username, password	Creates account	Yes — validation/rate limits
4	[GET] /identity/.../profile	Yes	token/cookie	Returns current user's profile	Yes — identity binding
5	[GET] /workshop/.../vehicles	Yes	pagination/filter parameters if present	Lists a user's vehicles	Yes — object ownership
6	[GET] /workshop/.../vehicles/{id}	Yes	vehicle/object ID	Returns one vehicle record	Yes — BOLA
7	[POST] /workshop/.../vehicles	Yes	vehicle attributes in body	Creates a vehicle record	No
8	[GET] /community/.../posts	Yes/verify	page, limit, search/filter	Lists community content	Yes — excessive data exposure
9	[POST] /identity/.../verify	Yes/verify	OTP/challenge ID/code	Completes a verification step	Yes — OTP workflow
10	[GET] /workshop/.../mechanic	Yes/verify	vehicle ID or service parameters	Service/repair workflow	Yes — business logic

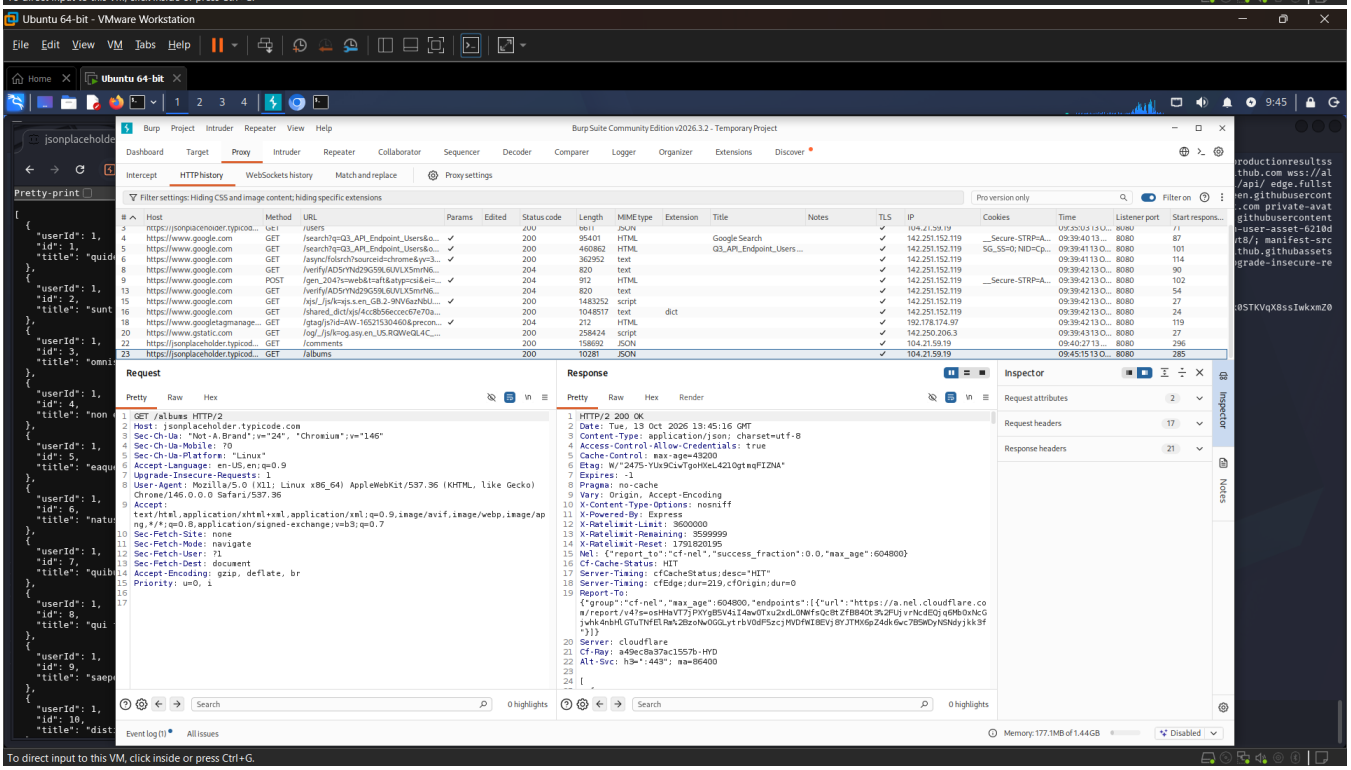
Important: Replace example paths with the exact paths observed in your authorized instance. If a path or function is not present, remove it and record a real endpoint instead. Authentication requirements should be verified by comparing a baseline request with and without the authorized session.

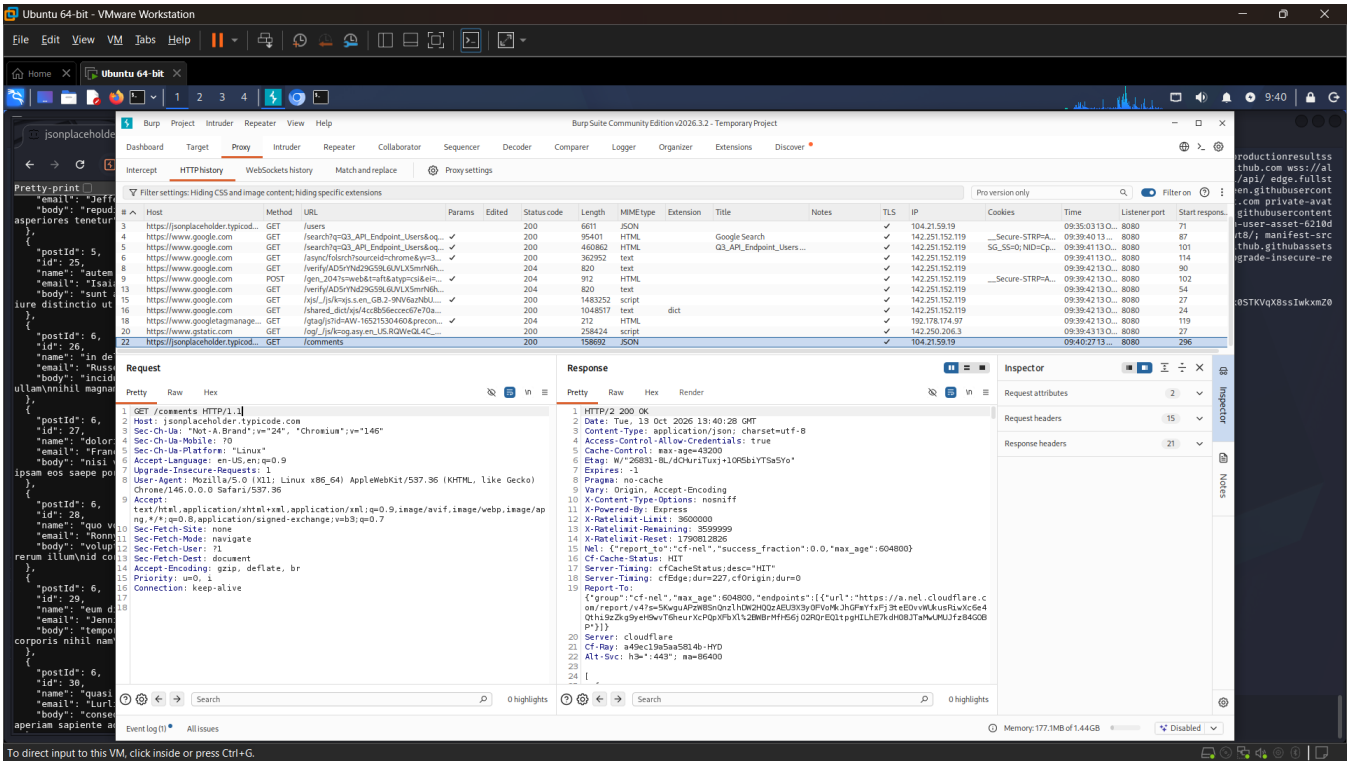
Reconnaissance procedure

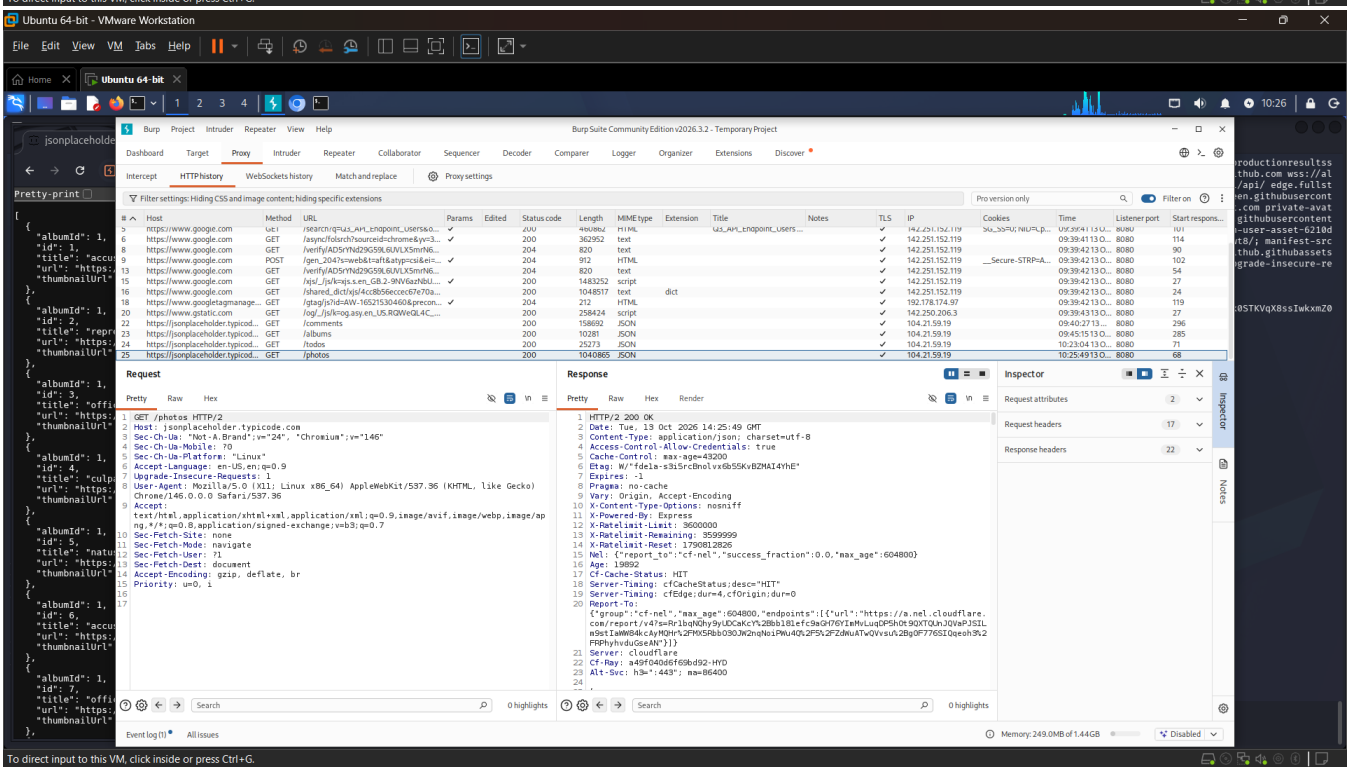
8. Open the approved crAPI web application and log in with a lab account.
9. Start Burp Suite and configure the browser to use the Burp proxy.
10. Perform ordinary app actions such as viewing a profile, listing vehicles, or opening a post; avoid actions that modify or delete data unless specifically required.
11. In HTTP history, record the method, path, status code, request parameters, and whether a cookie or bearer token is present.
12. Compare the endpoint list with the approved OpenAPI specification and remove duplicates.
13. Select at least three endpoints for deeper authorization testing, prioritizing endpoints that read or modify user-owned objects, expose profile data, or perform sensitive verification.

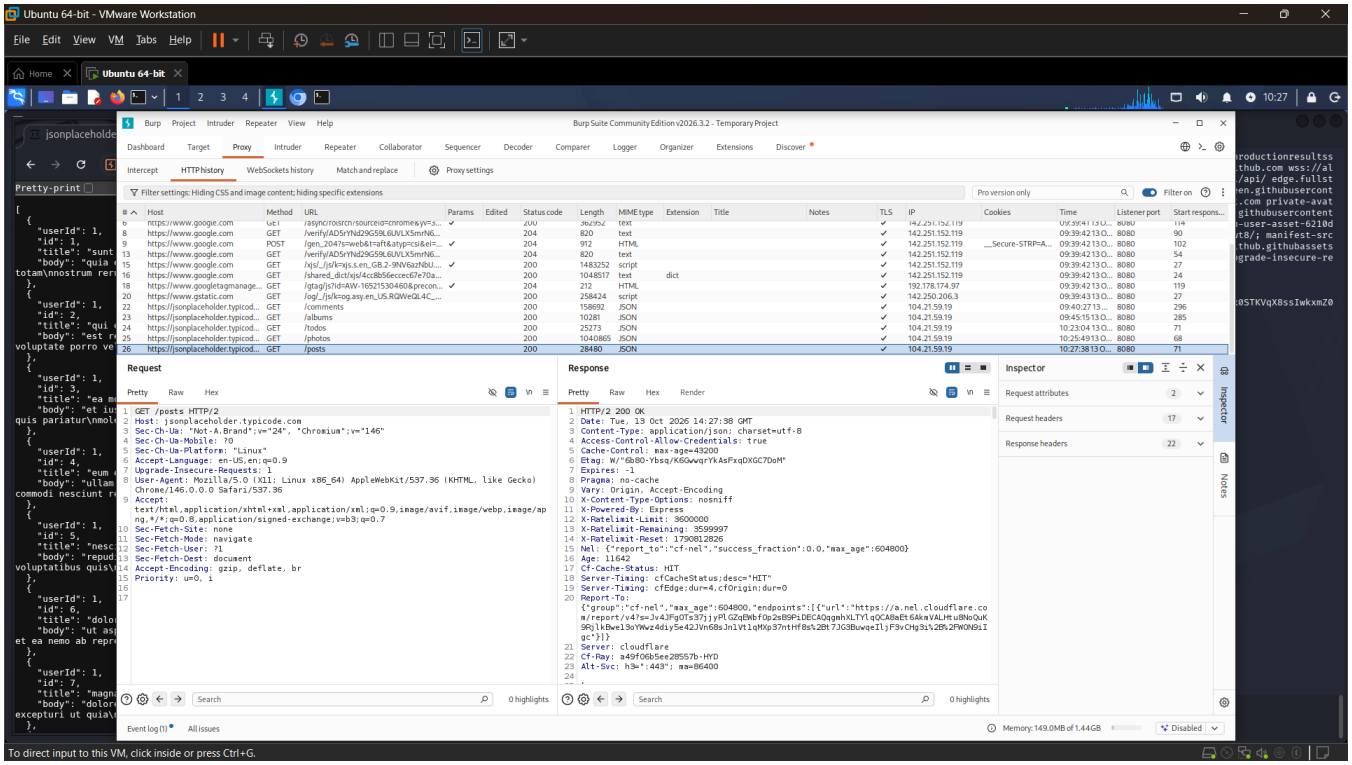
Evidence E-01A — Burp HTTP history showing discovered endpoints

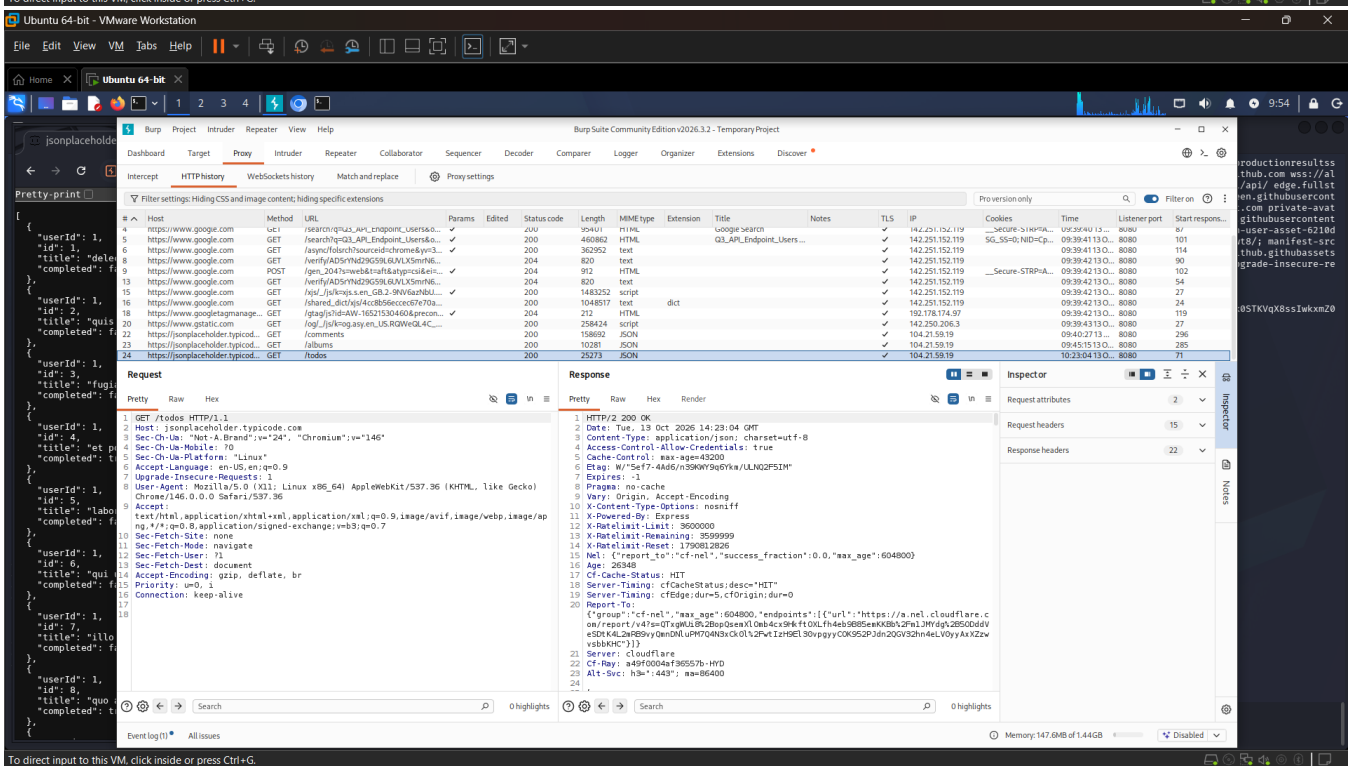


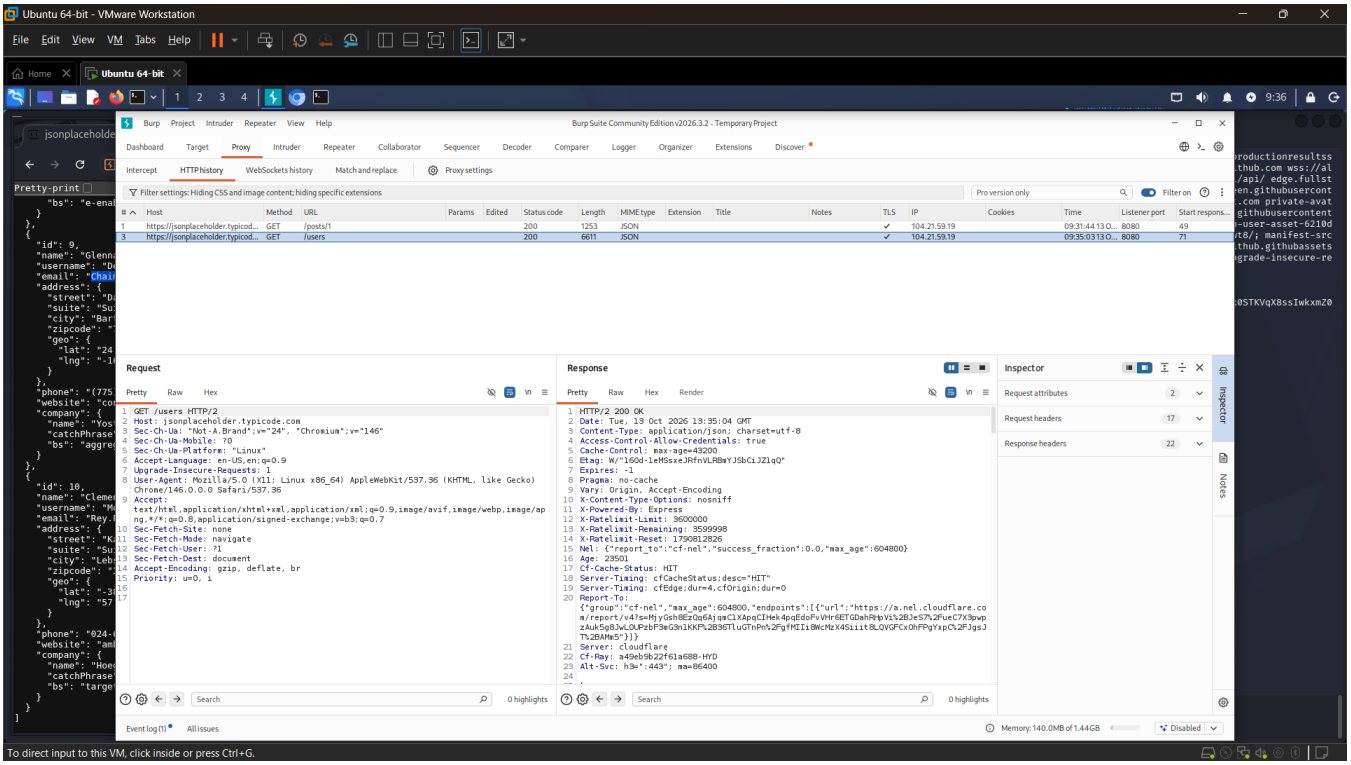


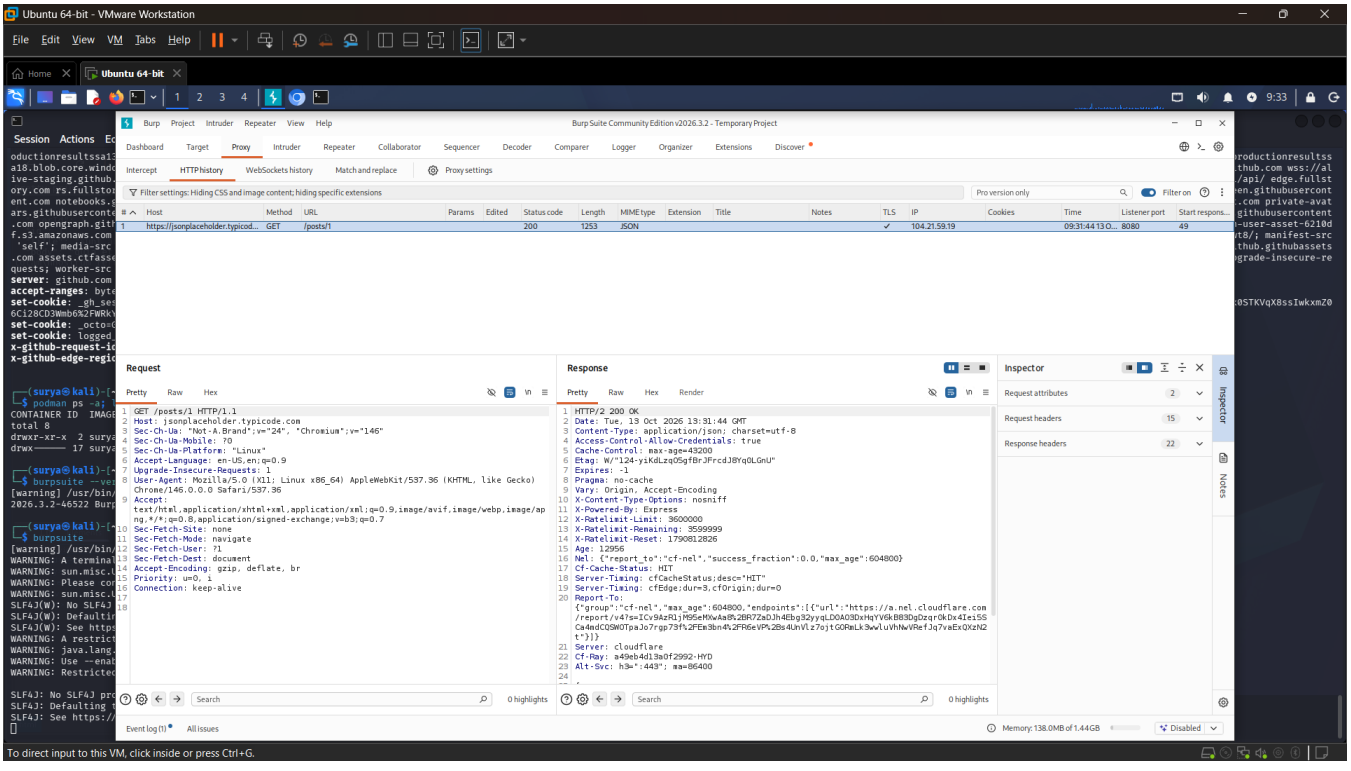

Ubuntu 64-bit - VMware Workstation
File Edit View VM Tabs Help
Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2026.3.2 - Temporary Project
Request
GET /comments HTTP/1.1
Host: jsonplaceholder.typicode.com
Response
HTTP/2 200 OK
Inspector
To direct input to this VM, click inside or press Ctrl+G.


Ubuntu 64-bit - VMware Workstation
File Edit View VM Tabs Help
Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2026.3.2 - Temporary Project
Request
GET /photos HTTP/2
Host: jsonplaceholder.typicode.com
Response
HTTP/2 200 OK
Inspector
To direct input to this VM, click inside or press Ctrl+G.

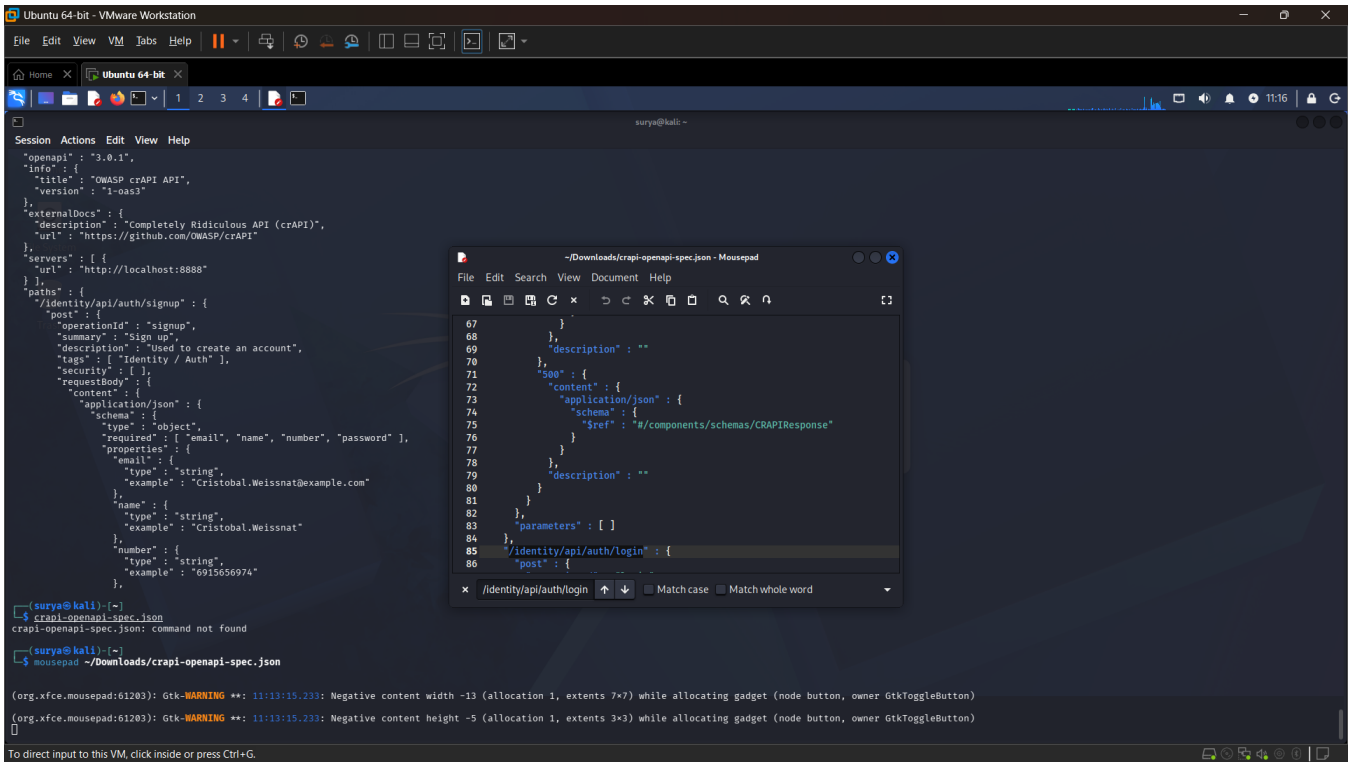

Ubuntu 64-bit - VMware Workstation
File Edit View VM Tabs Help
Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2026.3.2 - Temporary Project
Request
GET /posts HTTP/2
Host: jsonplaceholder.typicode.com
Response
HTTP/2 200 OK
Inspector
To direct input to this VM, click inside or press Ctrl+G.

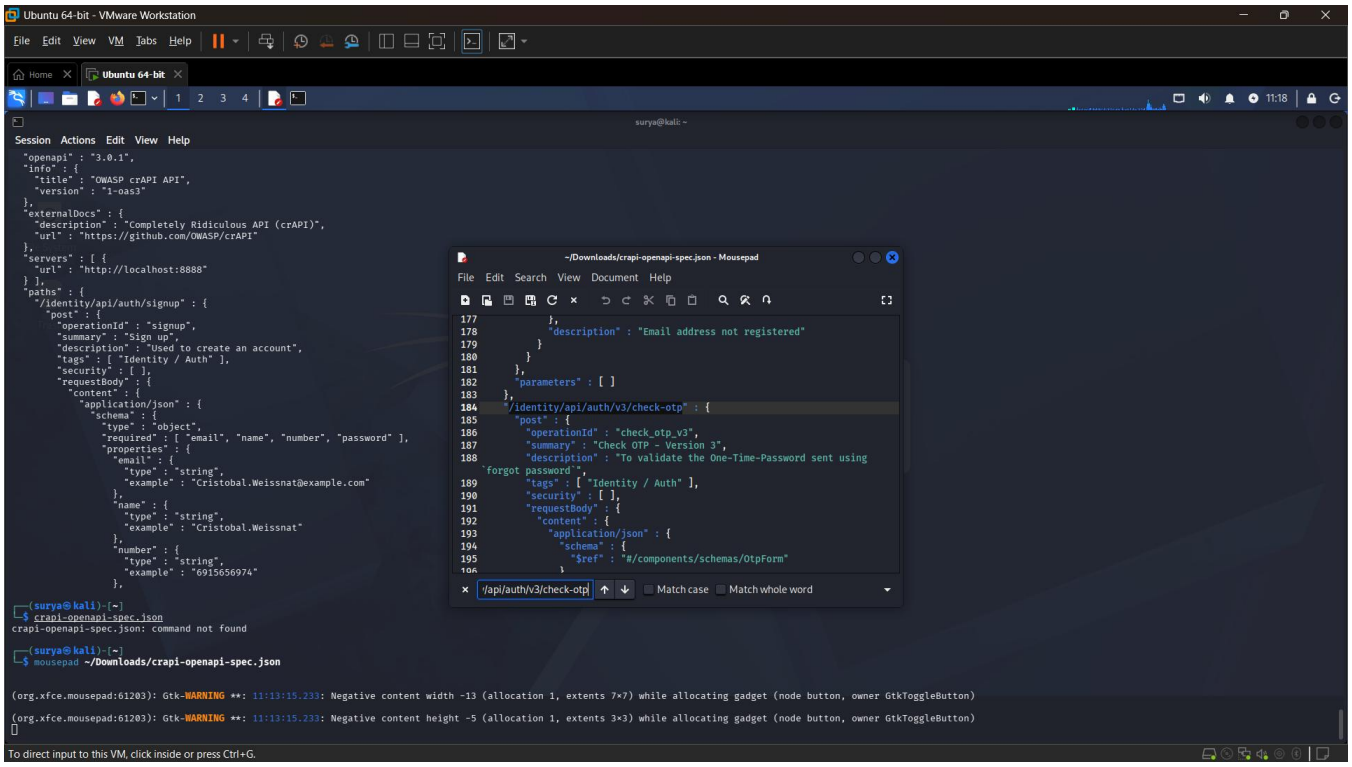

Ubuntu 64-bit - VMware Workstation
File Edit View VM Tabs Help
Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2026.3.2 - Temporary Project
Request
GET /todos HTTP/1.1
Host: jsonplaceholder.typicode.com
Response
HTTP/2 200 OK
Inspector
To direct input to this VM, click inside or press Ctrl+G.

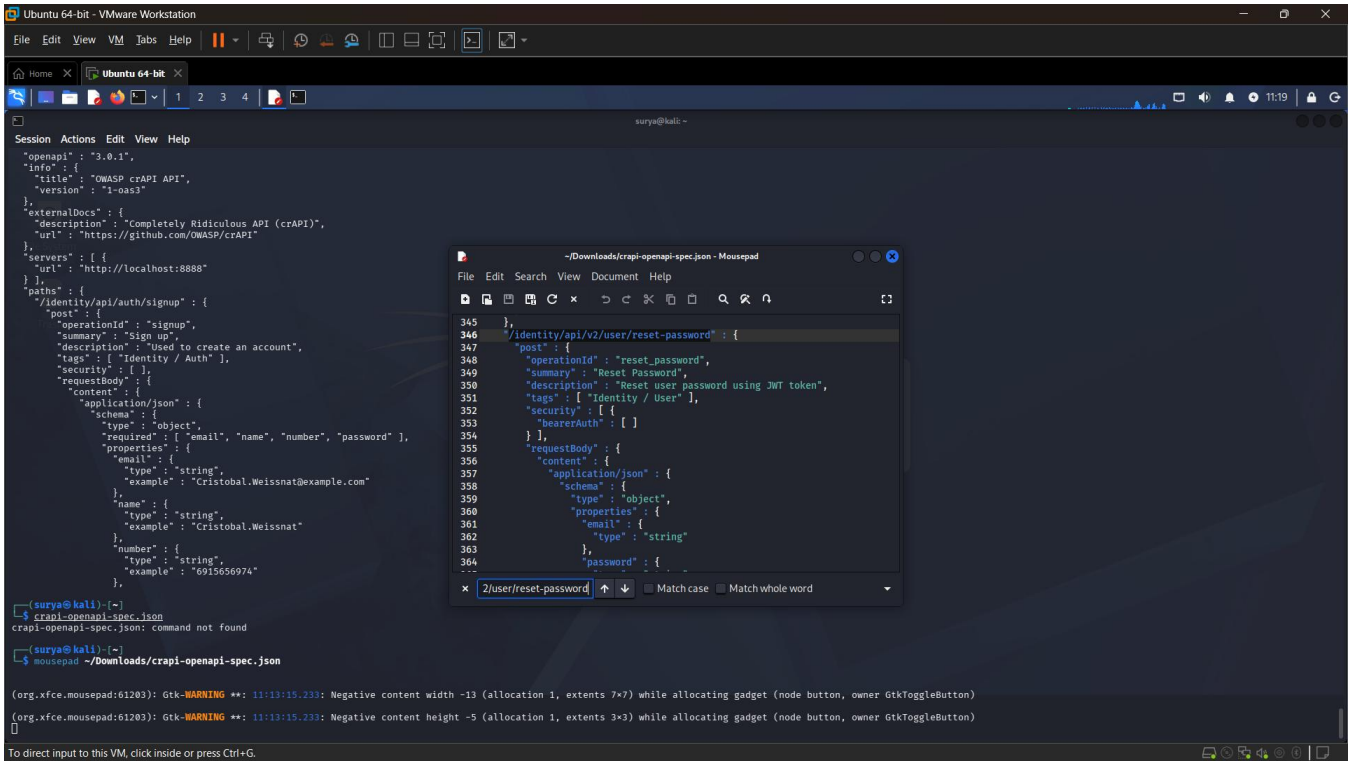

Ubuntu 64-bit - VMware Workstation
File Edit View VM Tabs Help
Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2026.3.2 - Temporary Project
Request
Response
Inspector
GET /users HTTP/2
HTTP/2 200 OK
To direct input to this VM, click inside or press Ctrl+G.

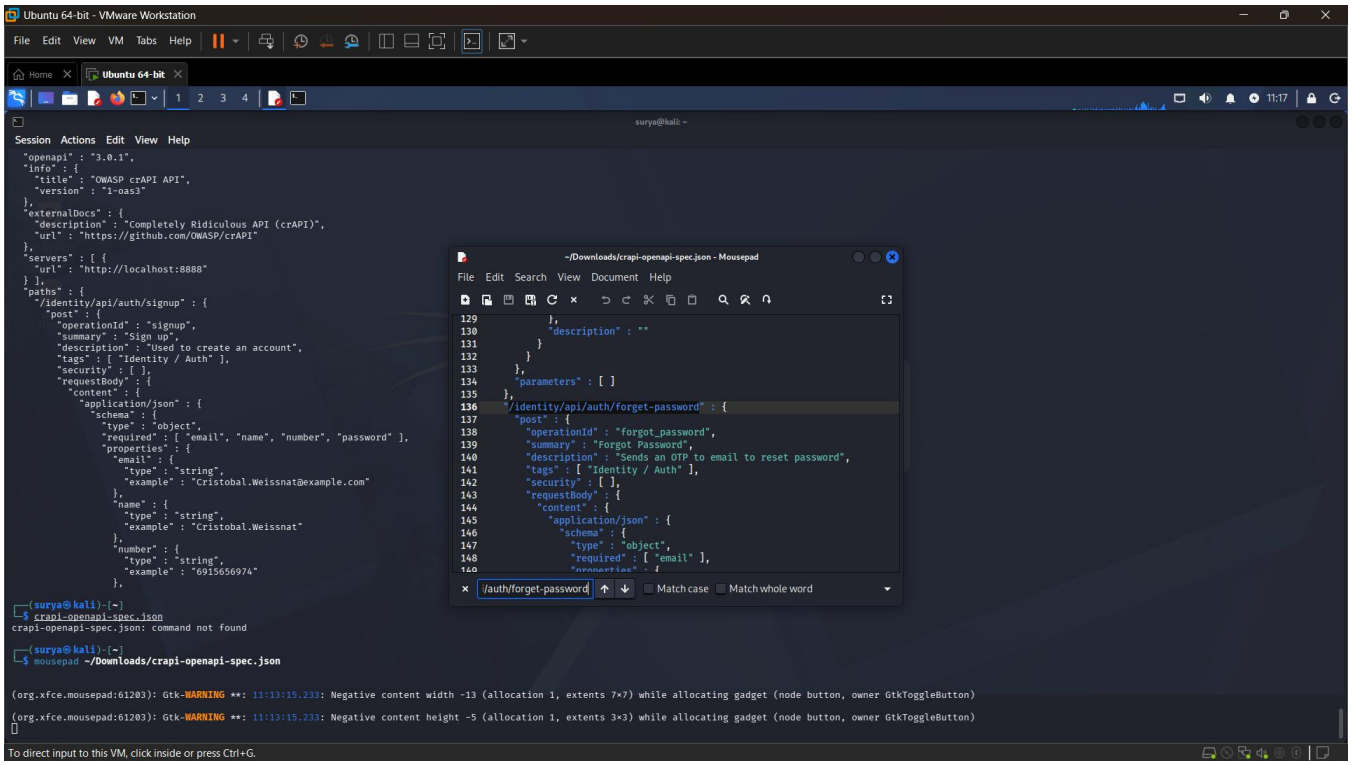

Ubuntu 64-bit - VMware Workstation
File Edit View VM Tabs Help
Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2026.3.2 - Temporary Project
Request
Response
Inspector
GET /posts/1 HTTP/1.1
HTTP/2 200 OK
To direct input to this VM, click inside or press Ctrl+G.

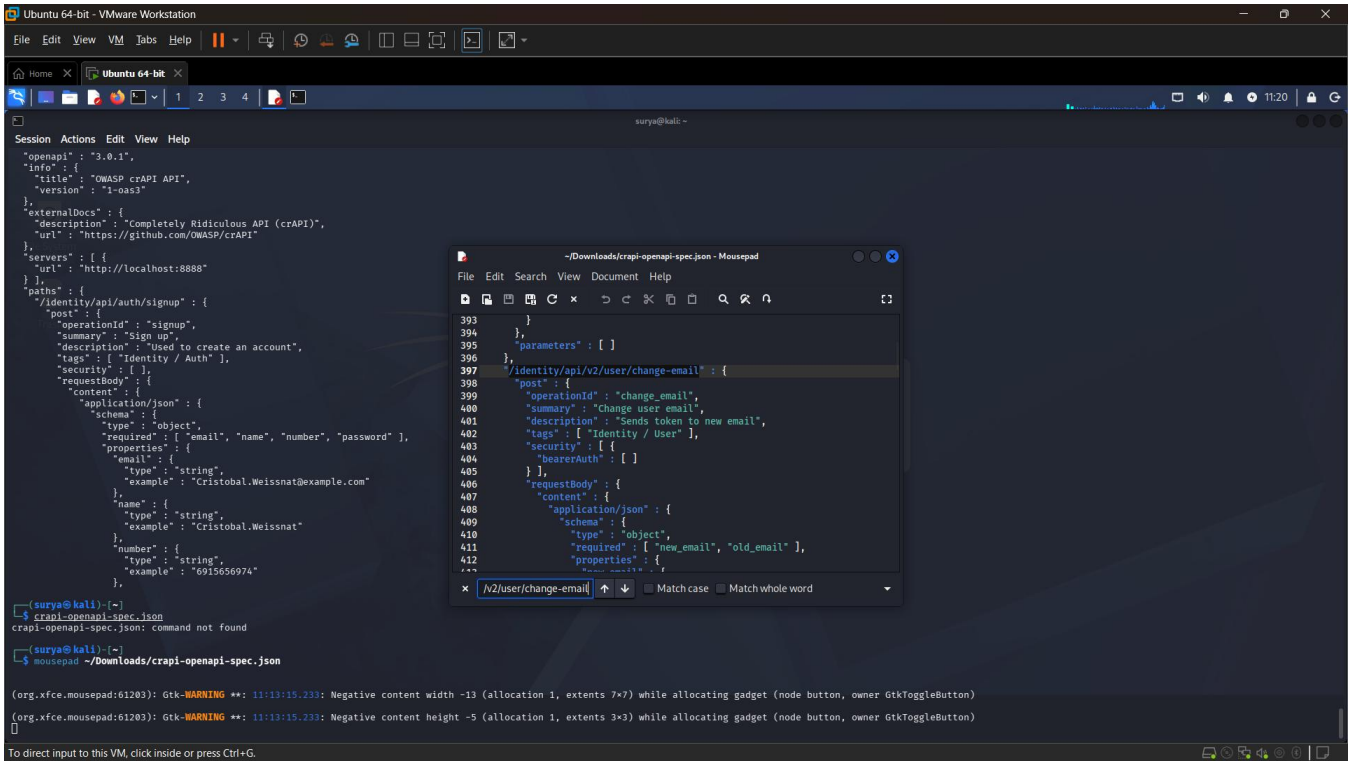
Evidence E-01B — API specification or endpoint list

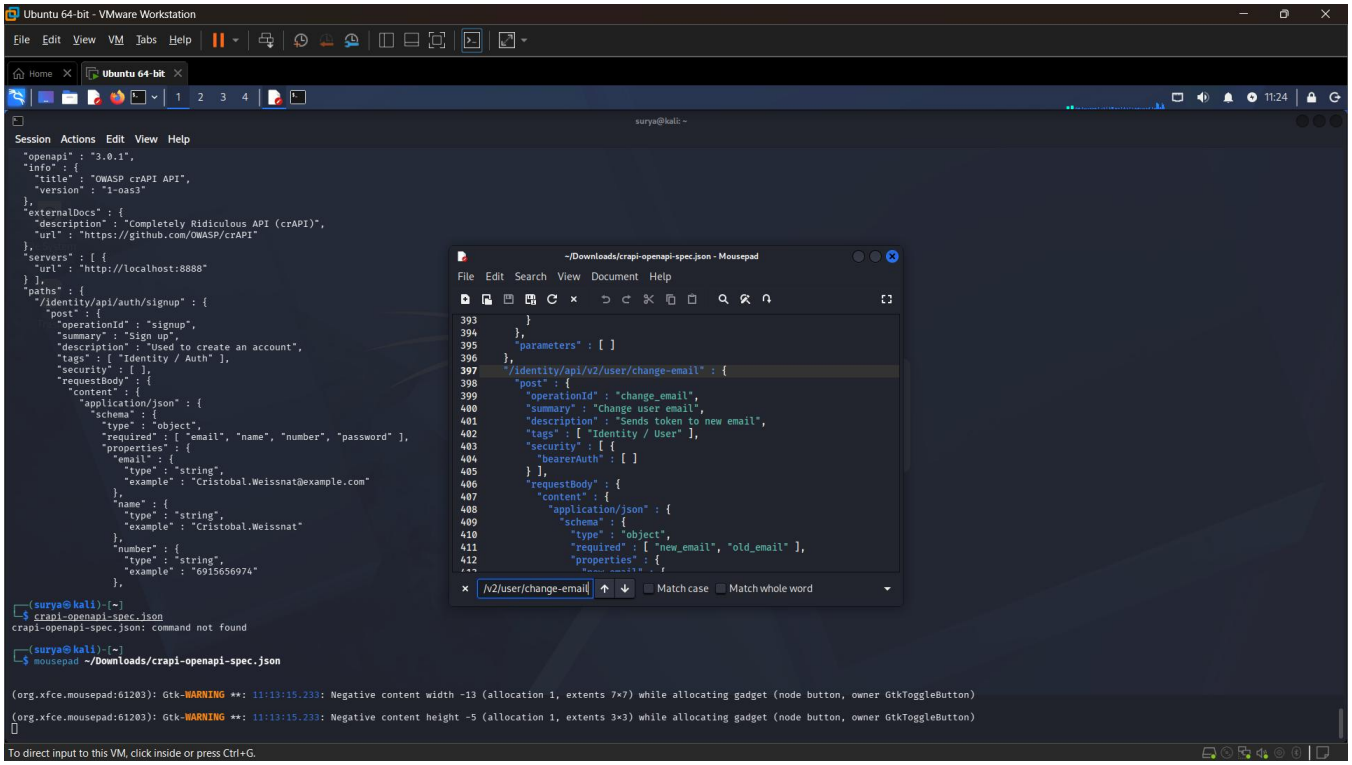


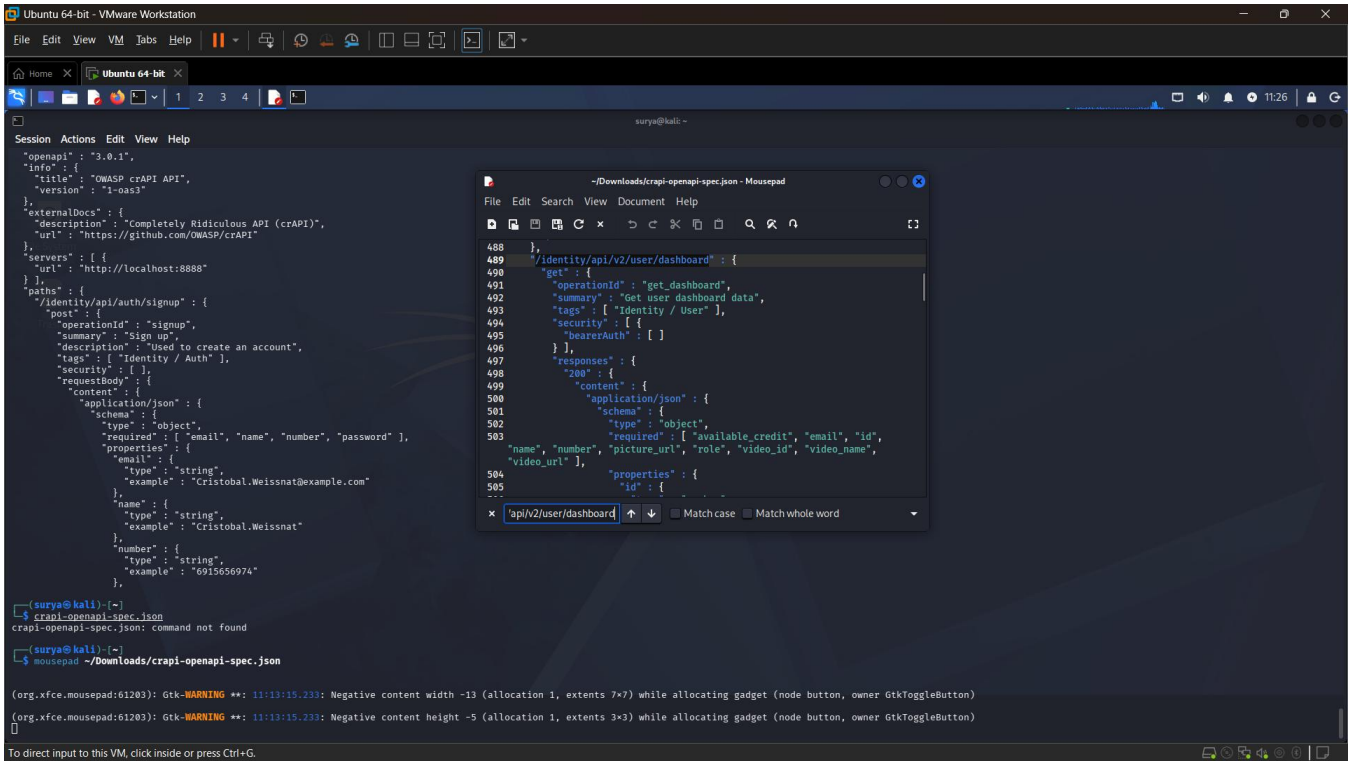


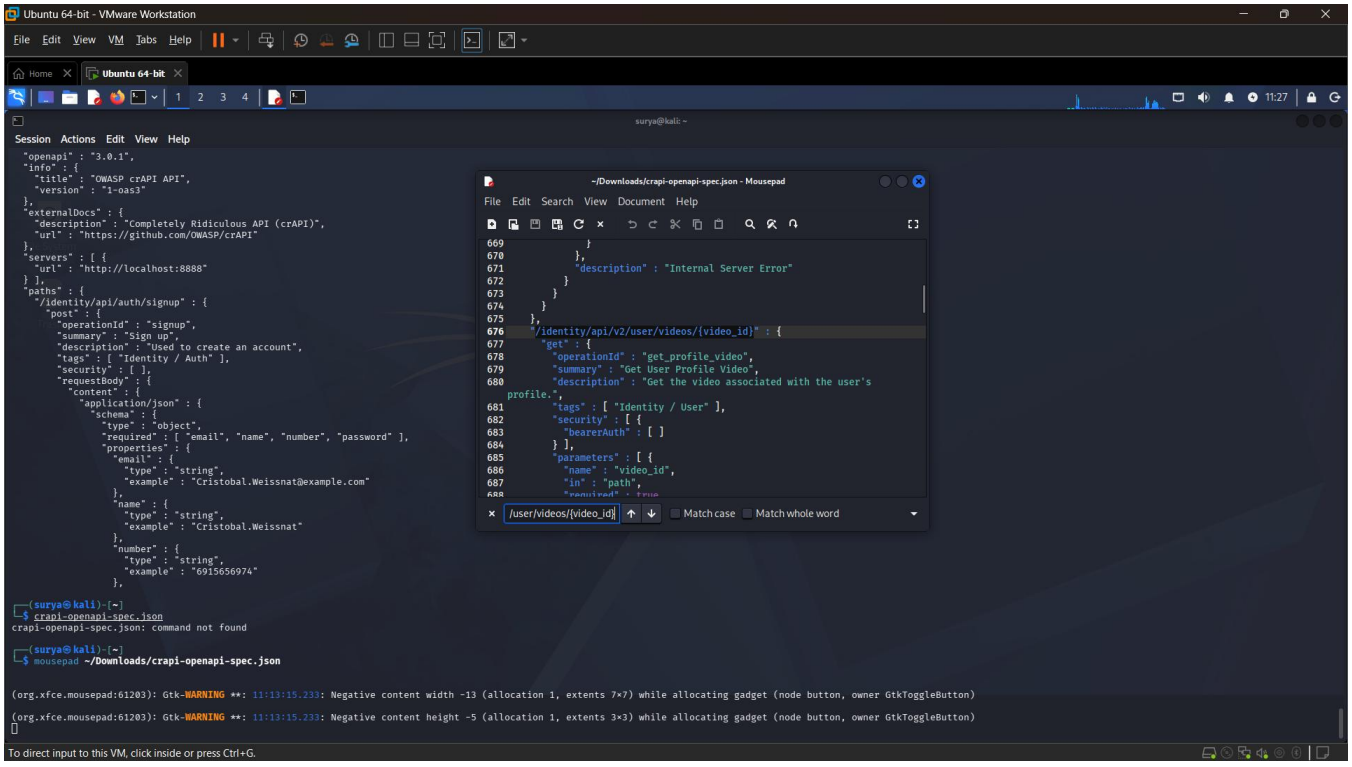

Ubuntu 64-bit - VMware Workstation
File Edit View VM Tabs Help
Home
Ubuntu 64-bit
surya@kali: ~
Session Actions Edit View Help
~/Downloads/crapi-openapi-spec.json - Mousepad
File Edit Search View Document Help
2/user/reset-password
Match case
Match whole word
(surya㉿kali)-[~]
$ crapi-openapi-spec.json
crapi-openapi-spec.json: command not found
(surya㉿kali)-[~]
$ mousepad ~/Downloads/crapi-openapi-spec.json
(org.xfce.mousepad:61203): Gtk-WARNING **: 11:13:15.233: Negative content width -13 (allocation 1, extents 7×7) while allocating gadget (node button, owner GtkToggleButton)
(org.xfce.mousepad:61203): Gtk-WARNING **: 11:13:15.233: Negative content height -5 (allocation 1, extents 3×3) while allocating gadget (node button, owner GtkToggleButton)
To direct input to this VM, click inside or press Ctrl+G.


Ubuntu 64-bit - VMware Workstation
File Edit View VM Tabs Help
Home
Ubuntu 64-bit
surya@kali: ~
Session Actions Edit View Help
~/Downloads/crapi-openapi-spec.json - Mousepad
File Edit Search View Document Help
/auth/forget-password
Match case
Match whole word
(surya㉿kali)-[~]
$ crapi-openapi-spec.json
crapi-openapi-spec.json: command not found
(surya㉿kali)-[~]
$ mousepad ~/Downloads/crapi-openapi-spec.json
(org.xfce.mousepad:61203): Gtk-WARNING **: 11:13:15.233: Negative content width -13 (allocation 1, extents 7×7) while allocating gadget (node button, owner GtkToggleButton)
(org.xfce.mousepad:61203): Gtk-WARNING **: 11:13:15.233: Negative content height -5 (allocation 1, extents 3×3) while allocating gadget (node button, owner GtkToggleButton)
To direct input to this VM, click inside or press Ctrl+G.


Ubuntu 64-bit - VMware Workstation
File Edit View VM Tabs Help
Home
Ubuntu 64-bit
11:20
surya@kali: ~
Session Actions Edit View Help
"openapi" : "3.0.1",
"info" : {
"title" : "OWASP crAPI API",
"version" : "1-oas3"
},
"externalDocs" : {
"description" : "Completely Ridiculous API (crAPI)",
"url" : "https://github.com/OWASP/crAPI"
},
"servers" : [{
"url" : "http://localhost:8888"
}],
"paths" : {
"/identity/api/auth/signup" : {
"post" : {
"operationId" : "signup",
"summary" : "Sign up",
"description" : "Used to create an account",
"tags" : ["Identity / Auth"],
"security" : [ ],
"requestBody" : {
"content" : {
"application/json" : {
"schema" : {
"type" : "object",
"required" : ["email", "name", "number", "password"],
"properties" : {
"email" : {
"type" : "string",
"example" : "Cristobal.Weissnat@example.com"
},
"name" : {
"type" : "string",
"example" : "Cristobal.Weissnat"
},
"number" : {
"type" : "string",
"example" : "6915656974"
},
~/Downloads/crapi-openapi-spec.json - Mousepad
File Edit Search View Document Help
393 }
394 },
395 "parameters" : [ ]
396 },
397 "/identity/api/v2/user/change-email" : {
398 "post" : {
399 "operationId" : "change_email",
400 "summary" : "Change user email",
401 "description" : "Sends token to new email",
402 "tags" : ["Identity / User"],
403 "security" : [{
404 "bearerAuth" : [ ]
405 }],
406 "requestBody" : {
407 "content" : {
408 "application/json" : {
409 "schema" : {
410 "type" : "object",
411 "required" : ["new_email", "old_email"],
412 "properties" : {
/v2/user/change-email
Match case
Match whole word
(surya㉿kali)-[~]
$ crapi-openapi-spec.json
crapi-openapi-spec.json: command not found
(surya㉿kali)-[~]
$ mousepad ~/Downloads/crapi-openapi-spec.json
(org.xfce.mousepad:61203): Gtk-WARNING **: 11:13:15.233: Negative content width -13 (allocation 1, extents 7×7) while allocating gadget (node button, owner GtkToggleButton)
(org.xfce.mousepad:61203): Gtk-WARNING **: 11:13:15.233: Negative content height -5 (allocation 1, extents 3×3) while allocating gadget (node button, owner GtkToggleButton)
To direct input to this VM, click inside or press Ctrl+G.


Ubuntu 64-bit - VMware Workstation
File Edit View VM Tabs Help
Home
Ubuntu 64-bit
11:24
surya@kali: ~
Session Actions Edit View Help
"openapi" : "3.0.1",
"info" : {
"title" : "OWASP crAPI API",
"version" : "1-oas3"
},
"externalDocs" : {
"description" : "Completely Ridiculous API (crAPI)",
"url" : "https://github.com/OWASP/crAPI"
},
"servers" : [{
"url" : "http://localhost:8888"
}],
"paths" : {
"/identity/api/auth/signup" : {
"post" : {
"operationId" : "signup",
"summary" : "Sign up",
"description" : "Used to create an account",
"tags" : ["Identity / Auth"],
"security" : [ ],
"requestBody" : {
"content" : {
"application/json" : {
"schema" : {
"type" : "object",
"required" : ["email", "name", "number", "password"],
"properties" : {
"email" : {
"type" : "string",
"example" : "Cristobal.Weissnat@example.com"
},
"name" : {
"type" : "string",
"example" : "Cristobal.Weissnat"
},
"number" : {
"type" : "string",
"example" : "6915656974"
},
~/Downloads/crapi-openapi-spec.json - Mousepad
File Edit Search View Document Help
393 }
394 },
395 "parameters" : [ ]
396 },
397 "/identity/api/v2/user/change-email" : {
398 "post" : {
399 "operationId" : "change_email",
400 "summary" : "Change user email",
401 "description" : "Sends token to new email",
402 "tags" : ["Identity / User"],
403 "security" : [{
404 "bearerAuth" : [ ]
405 }],
406 "requestBody" : {
407 "content" : {
408 "application/json" : {
409 "schema" : {
410 "type" : "object",
411 "required" : ["new_email", "old_email"],
412 "properties" : {
/v2/user/change-email
Match case
Match whole word
(surya㉿kali)-[~]
$ crapi-openapi-spec.json
crapi-openapi-spec.json: command not found
(surya㉿kali)-[~]
$ mousepad ~/Downloads/crapi-openapi-spec.json
(org.xfce.mousepad:61203): Gtk-WARNING **: 11:13:15.233: Negative content width -13 (allocation 1, extents 7×7) while allocating gadget (node button, owner GtkToggleButton)
(org.xfce.mousepad:61203): Gtk-WARNING **: 11:13:15.233: Negative content height -5 (allocation 1, extents 3×3) while allocating gadget (node button, owner GtkToggleButton)
To direct input to this VM, click inside or press Ctrl+G.


Ubuntu 64-bit - VMware Workstation
File Edit View VM Tabs Help
Home
Ubuntu 64-bit
11:26
surya@kali: ~
Session Actions Edit View Help
"openapi" : "3.0.1",
"info" : {
"title" : "OWASP crAPI API",
"version" : "1-oas3"
},
"externalDocs" : {
"description" : "Completely Ridiculous API (crAPI)",
"url" : "https://github.com/OWASP/crAPI"
},
"servers" : [{
"url" : "http://localhost:8888"
}],
"paths" : {
"/identity/api/auth/signup" : {
"post" : {
"operationId" : "signup",
"summary" : "Sign up",
"description" : "Used to create an account",
"tags" : ["Identity / Auth"],
"security" : [ ],
"requestBody" : {
"content" : {
"application/json" : {
"schema" : {
"type" : "object",
"required" : ["email", "name", "number", "password"],
"properties" : {
"email" : {
"type" : "string",
"example" : "Cristobal.Weissnat@example.com"
},
"name" : {
"type" : "string",
"example" : "Cristobal.Weissnat"
},
"number" : {
"type" : "string",
"example" : "6915656974"
},
(surya㉿kali)-[~]
$ crapi-openapi-spec.json
crapi-openapi-spec.json: command not found
(surya㉿kali)-[~]
$ mousepad ~/Downloads/crapi-openapi-spec.json
(org.xfce.mousepad:61203): Gtk-WARNING **: 11:13:15.233: Negative content width -13 (allocation 1, extents 7×7) while allocating gadget (node button, owner GtkToggleButton)
(org.xfce.mousepad:61203): Gtk-WARNING **: 11:13:15.233: Negative content height -5 (allocation 1, extents 3×3) while allocating gadget (node button, owner GtkToggleButton)
To direct input to this VM, click inside or press Ctrl+G.
~/Downloads/crapi-openapi-spec.json - Mousepad
File Edit Search View Document Help
},
"/identity/api/v2/user/dashboard" : {
"get" : {
"operationId" : "get_dashboard",
"summary" : "Get user dashboard data",
"tags" : ["Identity / User"],
"security" : [{
"bearerAuth" : [ ]
}],
"responses" : {
"200" : {
"content" : {
"application/json" : {
"schema" : {
"type" : "object",
"required" : ["available_credit", "email", "id",
"name", "number", "picture_url", "role", "video_id", "video_name",
"video_url"],
"properties" : {
"id" : {
'api/v2/user/dashboard
Match case
Match whole word


Ubuntu 64-bit - VMware Workstation
File Edit View VM Tabs Help
Home
Ubuntu 64-bit
11:27
surya@kali: ~
Session Actions Edit View Help
"openapi" : "3.0.1",
"info" : {
"title" : "OWASP crAPI API",
"version" : "1-oas3"
},
"externalDocs" : {
"description" : "Completely Ridiculous API (crAPI)",
"url" : "https://github.com/OWASP/crAPI"
},
"servers" : [{
"url" : "http://localhost:8888"
}],
"paths" : {
"/identity/api/auth/signup" : {
"post" : {
"operationId" : "signup",
"summary" : "Sign up",
"description" : "Used to create an account",
"tags" : ["Identity / Auth"],
"security" : [ ],
"requestBody" : {
"content" : {
"application/json" : {
"schema" : {
"type" : "object",
"required" : ["email", "name", "number", "password"],
"properties" : {
"email" : {
"type" : "string",
"example" : "Cristobal.Weissnat@example.com"
},
"name" : {
"type" : "string",
"example" : "Cristobal.Weissnat"
},
"number" : {
"type" : "string",
"example" : "6915656974"
},
(surya㉿kali)-[~]
$ crapi-openapi-spec.json
crapi-openapi-spec.json: command not found
(surya㉿kali)-[~]
$ mousepad ~/Downloads/crapi-openapi-spec.json
(org.xfce.mousepad:61203): Gtk-WARNING **: 11:13:15.233: Negative content width -13 (allocation 1, extents 7×7) while allocating gadget (node button, owner GtkToggleButton)
(org.xfce.mousepad:61203): Gtk-WARNING **: 11:13:15.233: Negative content height -5 (allocation 1, extents 3×3) while allocating gadget (node button, owner GtkToggleButton)
To direct input to this VM, click inside or press Ctrl+G.
~/Downloads/crapi-openapi-spec.json - Mousepad
File Edit Search View Document Help
}
},
"description" : "Internal Server Error"
}
}
}
},
"/identity/api/v2/user/videos/{video_id}" : {
"get" : {
"operationId" : "get_profile_video",
"summary" : "Get User Profile Video",
"description" : "Get the video associated with the user's
profile.",
"tags" : ["Identity / User"],
"security" : [{
"bearerAuth" : [ ]
}],
"parameters" : [{
"name" : "video_id",
"in" : "path",
/user/videos/{video_id}
Match case
Match whole word

Q1. API Reconnaissance & Attack Surface Mapping conclusion:

The API Reconnaissance and Attack Surface Mapping assessment was completed using the authorized OWASP crAPI lab. A minimum of eight API endpoints were identified and documented based on their HTTP methods, paths, authentication requirements, important parameters, and security functions. The assessment helped establish an overview of the application's API attack surface and identify endpoints requiring further security evaluation.

At least three endpoints were shortlisted for deeper authorization testing, particularly to investigate Broken Object Level Authorization (BOLA), unauthorized access to sensitive information, and improper access control. This reconnaissance provides a foundation for subsequent API security testing, including authorization validation, input validation, and security control assessment.

Overall, the assessment demonstrates the importance of systematically mapping API endpoints before performing detailed security testing. Further testing should verify the identified risks using authorized test accounts and controlled requests, with appropriate evidence documented for each confirmed finding.

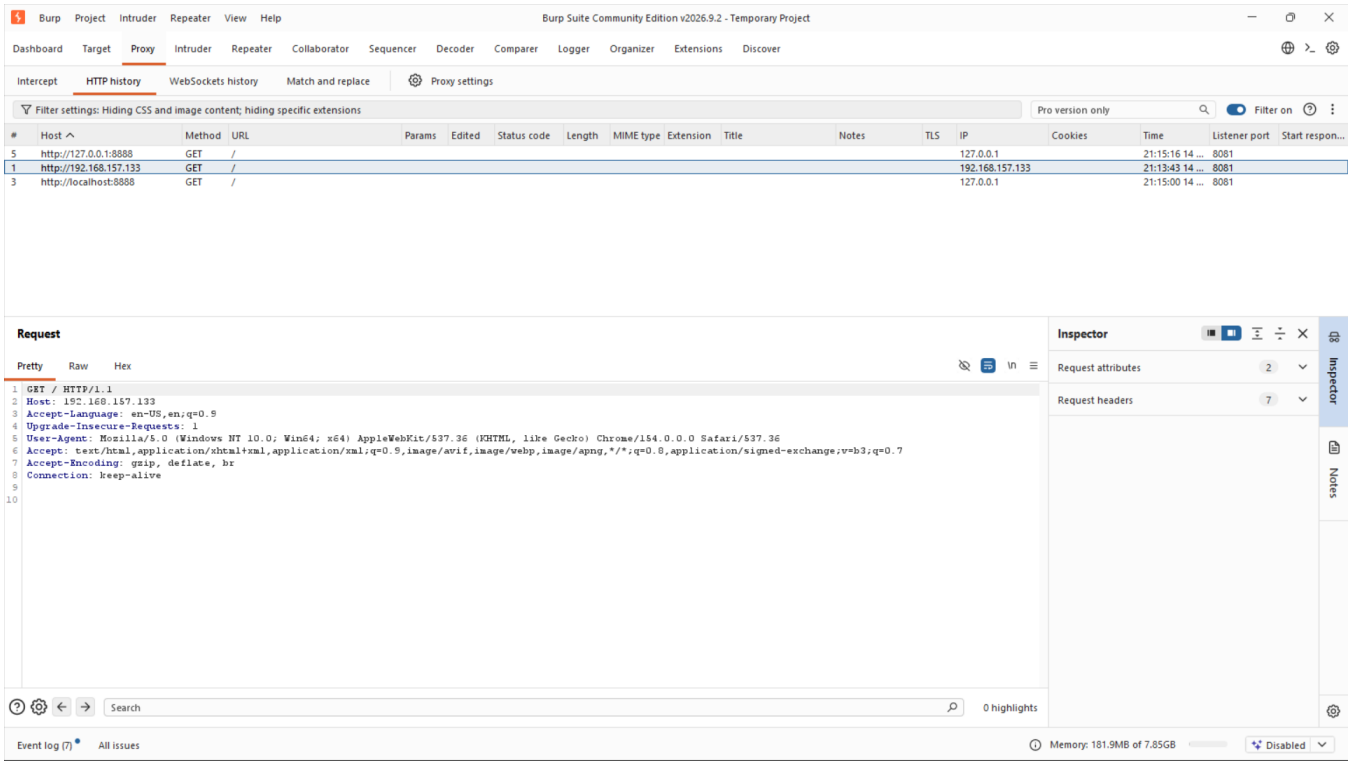
Q2. API Vulnerability Assessment (15 Marks)

Objective: Demonstrate two security weaknesses in the approved lab. Use the finding templates below. Choose only weaknesses supported by observed behavior. Examples include object-level authorization failure, excessive data exposure, weak input validation, or insecure business logic.

Finding F-01 — [Enter confirmed weakness]

Field	Report content
Title / Severity	[Example title; assign severity only after confirming impact]
Affected endpoint	[Exact HTTP method and path from Burp]
Description	[Explain the expected control and the observed difference]
Preconditions	[Lab account/role and object created for this test]
Reproduction steps	1. Send a normal baseline request. 2. Change one test input only. 3. Send the request again. 4. Compare status, response fields, and server-side effect.
Evidence	[Evidence ID, screenshot, sanitized request/response; remove tokens and personal data]
Impact	[Describe what an attacker could access or change, limited to what was demonstrated]
Remediation	Enforce server-side authorization and input validation; apply least privilege; return only required fields; add regression tests.
Retest criteria	Repeat the same request with authorized and unauthorized test accounts. The unauthorized action must be denied and no protected data or state change returned.

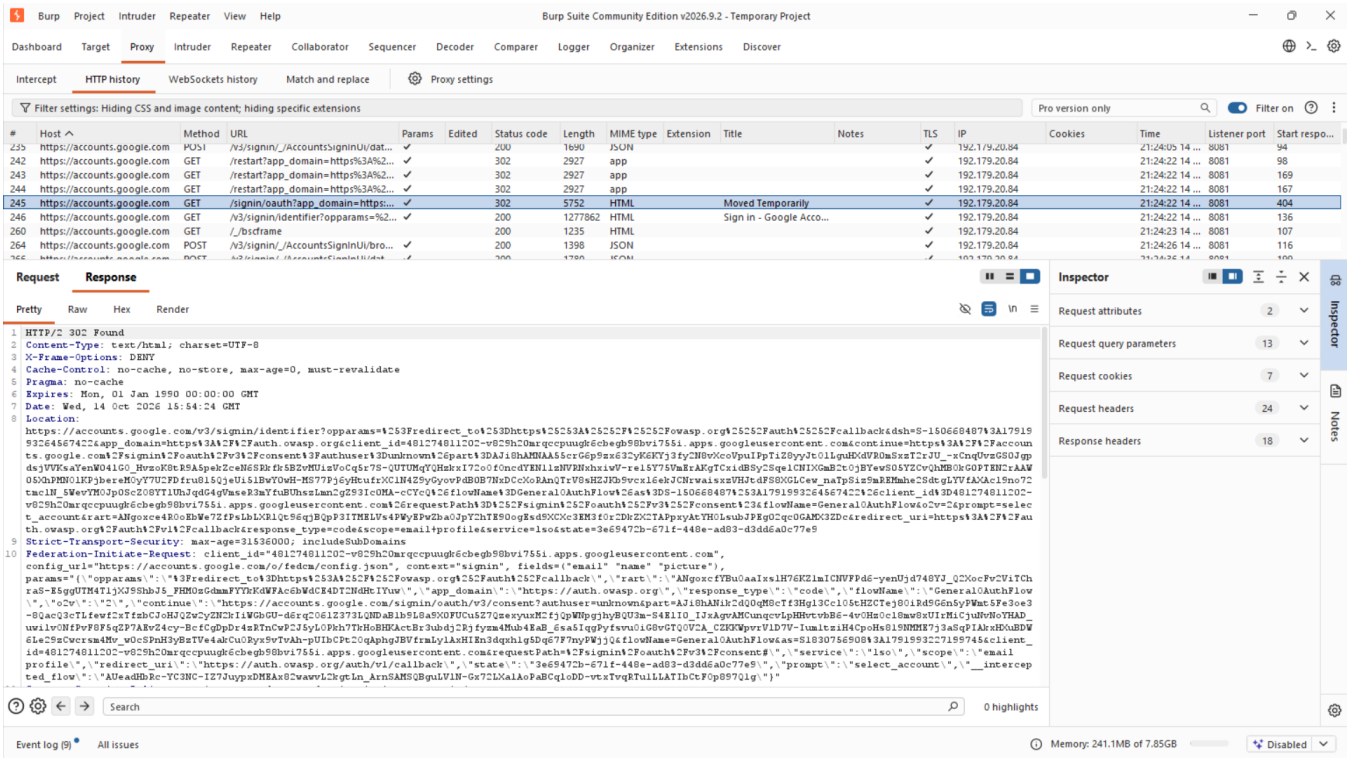
Evidence E-02A — Finding F-01 request/response



Finding F-02 — [Enter confirmed weakness]

Field	Report content
Title / Severity	[Example title; assign severity only after confirming impact]
Affected endpoint	[Exact HTTP method and path from Burp]
Description	[Expected control versus actual behavior]
Preconditions	[Lab account, role, and required setup]
Reproduction steps	1. Capture the baseline. 2. Change one input or workflow step. 3. Repeat using the authorized lab. 4. Record the smallest response that proves the issue.
Evidence	[Evidence ID and sanitized request/response]
Impact	[Confidentiality, integrity, or availability impact demonstrated]
Remediation	[Specific server-side control, validation, or workflow correction]
Retest criteria	[State the expected secure response and confirm the issue no longer reproduces]

Evidence E-02B — Finding F-02 request/response



If a weakness cannot be reproduced, document it as 'Not reproduced' or 'Potential concern' rather than a confirmed vulnerability. Do not invent status codes, response contents, or screenshots.

Q3. crAPI Practical Investigation (10 Marks)

Selected attack path: Object-level authorization check in a user-owned resource workflow (use this only if supported by your authorized crAPI version). This investigation examines whether a request for a resource is authorized against the logged-in user's identity, rather than trusting a client-supplied object ID.

Investigation flow

14. Discovery: use the application normally and identify an endpoint that returns a resource belonging to the lab account.
15. Baseline: save a request for an object owned by Test User A and confirm the expected response.
16. Controlled comparison: create or use a second lab-owned object belonging to Test User B. Do not enumerate IDs or use real users.
17. Authorization check: while authenticated as User A, request only the known test object belonging to User B.
18. Observe: compare status code, response fields, and any state change. A different ID alone is not proof; the key question is whether the server checks ownership.
19. Impact: record only the minimum evidence needed to show whether cross-account access is allowed. Stop immediately after confirmation.
20. Remediation: enforce ownership/role checks server-side for every object read or write.

Stage	Request / parameter	Expected behavior
1. Baseline	[Method/path; own object ID]	Authorized object returned
2. Comparison	[Same request; known second test user's object ID]	Denied or safely not found
3. Conclusion	[No additional requests required]	No cross-user data access

SECTION B — BROKEN ACCESS CONTROL / BOLA

Q4. BOLA Scenario (10 Marks)

Scenario: An authenticated User A sends GET /api/users/102/profile. Changing the object ID to another valid ID returns another user's profile. This is a strong indicator of Broken Object Level Authorization (BOLA) if the server returns an object that User A is not permitted to access.

Safe verification

21. Use only an instructor-approved lab and two accounts you control.
22. Capture the baseline request for User A's own profile/object, including method, path, status code, and relevant response fields.
23. Use a known test object ID belonging to User B; do not guess, scan, or enumerate IDs.
24. Repeat the request while keeping User A's session unchanged and changing only the object ID.
25. Verify whether protected fields are returned. If the response contains sensitive content, record only a redacted excerpt and stop.
26. Repeat once if needed to rule out caching or a transient error; do not access more records than necessary.

Evidence to collect

- A sanitized baseline request and response for User A's own object.
- A sanitized comparison request showing the changed test object ID.
- The response status and only the minimum field(s) that demonstrate access was allowed or denied.
- The account/role used, timestamp, endpoint, and expected versus actual behavior.
- A note confirming that both objects and accounts were created or supplied for the authorized lab.

Business impact

Depending on the object type, BOLA can expose personal information, order details, vehicle records, account settings, or other private data. If write operations are also affected, an attacker may modify or delete another user's resources. State only the impact supported by the test.

Recommended fix

The server must derive the authenticated principal from a validated session/token and check that principal's relationship to the requested object on every request. Do not rely on an object ID being difficult to guess or on the client hiding links. Apply authorization to read, update, and delete operations, return minimal fields, and add automated cross-user regression tests.

Q5. Authorization Testing Matrix (10 Marks)

Use two lab accounts with different ownership/privileges. Create a small number of test objects with harmless values. Execute only the actions permitted by the lab and avoid testing delete operations on anything that is not disposable.

Test	User A action against A object	User A action against B object	User B action against B object	Expected secure outcome
Read	GET own object	GET B object	GET own object	A can read A; A cannot read B; B can read B
Modify	PATCH/PUT own test object	PATCH/PUT B test object	PATCH/PUT own test object	Only the owner/authorized role can modify
Delete	DELETE disposable A test object	DELETE disposable B test object	DELETE disposable B test object	Only the owner/authorized role can delete; use disposable data only

Distinguishing authentication from authorization

Observation	Likely interpretation	How to verify safely
401 Unauthorized / missing or invalid token	Authentication failed or credentials were not accepted	Repeat with a valid lab session; do not infer BOLA from this result alone.
403 Forbidden for a valid session	Request is authenticated but permission is denied	Compare a user who legitimately owns the object.
404 Not Found	May mean missing object or intentionally concealed unauthorized object	Compare only known lab objects; note that 404 alone does not prove secure authorization.
200 OK with another user's protected object	Potential BOLA if access is not permitted	Confirm using two controlled accounts and minimal redacted evidence.
200 OK but no protected fields / generic response	May be filtered or intentionally limited	Inspect only relevant fields and verify the intended access policy.

SECTION C — OTP / AUTHENTICATION BYPASS

Q6. OTP Bypass Investigation (10 Marks)

Objective: determine whether OTP verification is enforced by the server or merely by client-side application logic. Use only a test account and the approved application. Do not target real accounts, intercept real OTPs, or attempt high-volume guessing.

Normal workflow mapping

27. Start from a fresh test session and record the normal login or sensitive-action workflow.
28. Identify the request that starts OTP delivery/challenge creation and record the challenge identifier and expiry behavior, if visible.
29. Submit a valid test OTP and record the server response and resulting state/session.
30. Repeat with an intentionally incorrect OTP and verify that the server rejects it.
31. In a controlled test, omit the OTP field or alter only a non-secret workflow flag/parameter to check whether the server improperly accepts a skipped verification step. Do not brute-force codes.
32. Check whether a challenge can be reused after successful verification and whether an expired challenge is rejected, using only a small number of controlled attempts.
33. Stop once the minimum evidence establishes the behavior. If bypass is suspected but not proven, report it as a potential issue and explain the limitation.

Server-side controls expected

- Verification must be performed on the server and bound to the correct user, purpose, session, and challenge.
- The server must require a valid OTP/challenge before issuing a fully authenticated session or completing the protected action.
- OTP values must expire quickly and be single-use.
- Apply per-account, per-challenge, and risk-aware attempt limits with cooldowns; avoid revealing whether a particular account exists.
- Invalidate the challenge after success, expiry, excessive failures, or relevant session changes.
- Use secure session rotation and issue tokens only after the required authentication steps have succeeded.

Test case	Safe input	Expected secure behavior
Normal valid flow	Correct OTP for test account	Verification succeeds once
Invalid OTP	Incorrect test value	Rejected; no authenticated state
OTP omitted	Remove OTP field in lab request	Rejected; no verification bypass
Expired OTP	Use an expired test challenge	Rejected
Replay	Resubmit previously successful challenge once	Rejected as already used

Impact and remediation

If the server allows a user to bypass OTP, an attacker who has obtained a password or an incomplete session may reach an account or sensitive action without the required second factor. Severity depends on which account or action is protected and what was demonstrated. Fix the server-side state machine, enforce challenge binding and single use, expire challenges, rate-limit attempts, and prevent token issuance before verification.

Q7. Authentication Security Review (5 Marks)

Control	Explanation
1. Server-side validation	The server independently validates the OTP and challenge state. Client-side flags, hidden buttons, and JavaScript checks are not security boundaries.
2. Expiry	Each OTP/challenge has a short validity period; expired codes are rejected even if submitted correctly.
3. Attempt limits	Apply per-challenge and per-account rate limits, progressive delays or temporary lockouts, monitoring, and alerts for repeated failures.
4. Replay protection	Mark a challenge as consumed after success and reject duplicate use; bind the challenge to the account and intended operation.
5. Secure session/token handling	Do not issue a full session/token before OTP success. Rotate session identifiers after authentication, protect tokens in transit and storage, and invalidate relevant sessions on logout or risk events.

SECTION D — ANDROID APK & JADX ANALYSIS

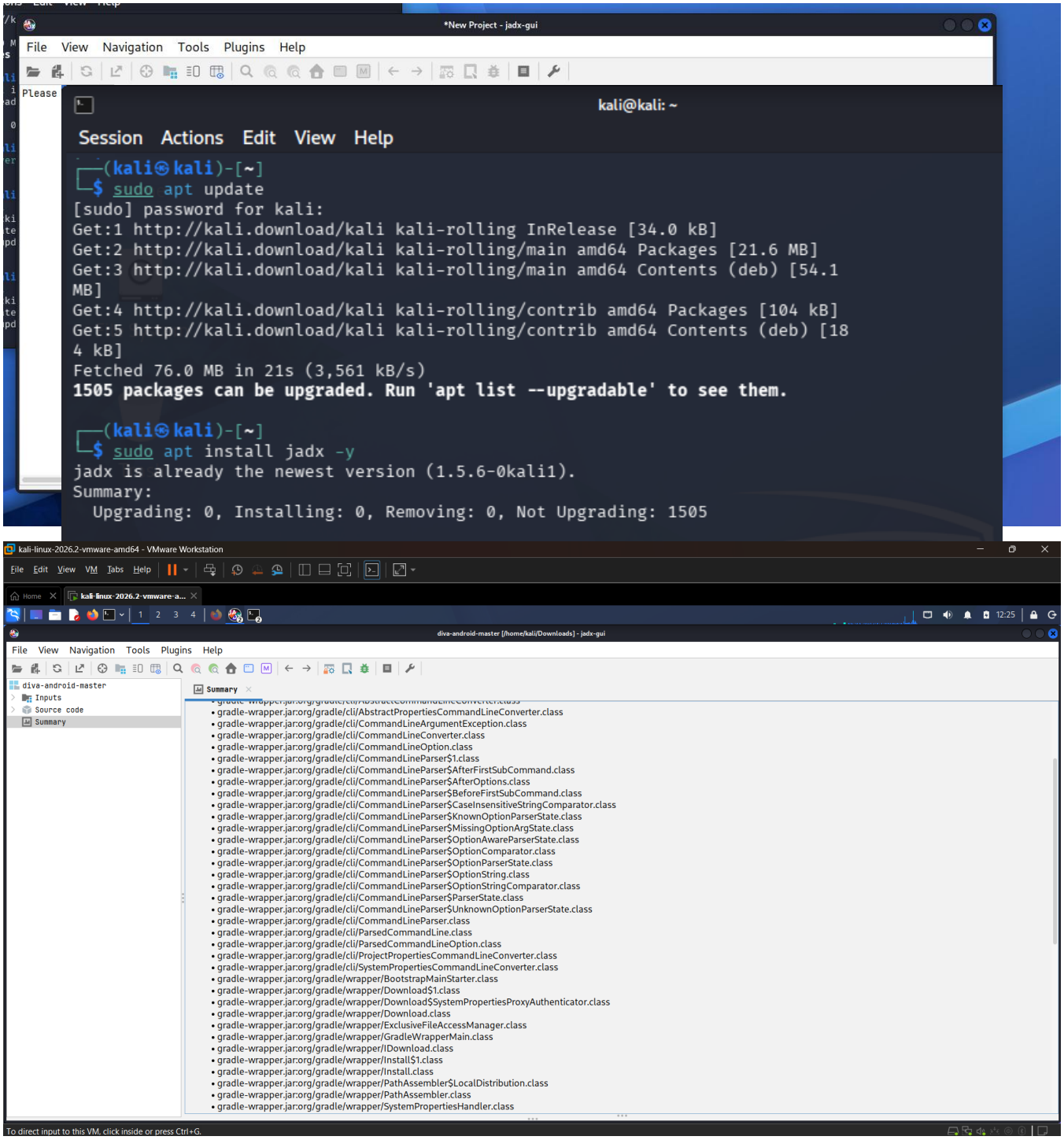
Q8. APK Reconnaissance with JADX (10 Marks)

Objective: statically inspect an authorized APK to understand its components, network destinations, configuration, and potential sensitive-data exposure. Static findings are leads; confirm whether a value is truly secret and active only through an approved, non-abusive validation process.

Procedure

34. Record the APK filename, source, file size, and SHA-256 hash. Preserve the original file unchanged.
35. Open JADX GUI and load the authorized APK. Allow decompilation to complete.
36. Inspect the AndroidManifest.xml view for package name, activities, services, receivers, providers, exported components, and permissions.
37. Search decompiled source and resources for strings such as https://, api, baseUrl, token, secret, key, client_id, password, Firebase, and debug.
38. Review interesting classes/packages that handle networking, authentication, storage, WebViews, deep links, and logging.
39. Record the source location and why each item matters. Redact the actual value of any suspected secret in the report.
40. Do not use exposed credentials to access a live service. Ask the instructor or service owner to validate and rotate any suspected credential.

Finding	Location / class	Sensitivity	Observation / evidence	Recommended action
Application components	AndroidManifest.xml	Low–Medium	[package, exported components, permissions]	Minimize exported components; require permissions/auth checks for entry points.
API endpoint	[class/resource file]	Low–Medium	[host/path only; no tokens]	Document intended host; remove test/debug endpoints from release builds.
Configuration value	[BuildConfig/resources]	Low–Medium	[value type, not sensitive value]	Review whether it is public configuration or a secret.
Possible hardcoded secret	[class and line/constant name]	High if valid	[redacted value, validation status unknown]	Revoke/rotate if valid; move privileged secrets to server-side secret storage.
Sensitive logging/storage	[class/method]	Medium–High	[what type of data is logged/stored]	Avoid logging tokens/PII; use platform-backed secure storage where appropriate.



Interpretation notes

An API base URL is usually not a secret by itself. A client identifier may be intended to be public. A privileged API key, private signing key, password, or reusable bearer token is more serious. Determine the intended privilege and exposure context before assigning severity. A value found in decompiled code should not be copied into the report in full.

Q9. APK Leak Scenario (5 Marks)

To determine whether a token-like value is sensitive and still valid, first identify its apparent purpose from the surrounding class, variable name, request headers, and documentation. Treat the value as compromised while its status is unknown. Do not submit it to production endpoints or use it to retrieve data.

- Check source/build configuration and owner-provided documentation to determine whether the value is a public client identifier or a privileged credential.
- Ask the instructor/service owner to validate the value in a dedicated test environment, or use an approved credential inventory/status check that does not access protected data.
- If validation is not explicitly authorized, record the value as 'potential secret — validity unverified' and stop.
- Assess potential impact by the permissions the credential is intended to grant, not by attempting unauthorized operations.
- Recommend revocation/rotation if active, reviewing logs for misuse, removing the secret from the source/build pipeline, and storing privileged credentials server-side in a managed secret store.
- Remember that deleting a value from the latest APK does not remove it from previously distributed APKs or repository history; rotate the exposed credential.

Potential impact: unauthorized API calls, data exposure, account abuse, quota theft, or service manipulation, depending on the credential's privilege. Severity should remain provisional until the value's sensitivity and reach are confirmed.

SECTION E — PENTESTER DECISION-MAKING

Q10. Integrated Scenario (10 Marks)

Scenario: An authorized mobile application contains a hidden API endpoint. The endpoint accepts an object ID, and changing that ID appears to expose another user's data. The APK also contains suspicious client-side OTP logic. The following plan prioritizes safety, evidence quality, and business impact.

Recommended testing order

Order	Action	Evidence to capture	Safety control / stop condition
1. Confirm scope	Confirm APK, API host, test accounts, and allowed test actions with the owner/instructor.	Scope note, APK filename/hash, target environment.	Do not test an unapproved production host.
2. Static APK review	Use JADX to locate the endpoint, network client, and OTP-related classes.	Manifest, class/package name, endpoint host/path; redact secrets.	Do not call discovered endpoints with embedded credentials.
3. Map normal API behavior	Use Burp to capture an ordinary request from a lab account.	Baseline request/response, role, object ownership, status.	Mask cookies/tokens; avoid unrelated data.
4. Verify BOLA minimally	Compare User A's own test object with one known User B test object.	Two sanitized requests and minimal response fields.	No ID enumeration; stop after one clear proof.
5. Review OTP flow	Map normal, invalid, omitted, expired, and one replay case using a test account.	Challenge flow and server responses.	No brute force or real account testing.
6. Rate and report	Assess confidentiality/integrity impact, likelihood, affected users, and exploit prerequisites.	Finding records, severity rationale, remediation.	Mark unconfirmed issues as potential/inconclusive.
7. Retest and close	Repeat the smallest proof after remediation using the same test objects.	Before/after evidence and retest outcome.	Clean up only test data you created, per lab instructions.

Evidence handling

- Use unique evidence IDs and include date/time, tool, account role, endpoint, and test case.
- Redact Authorization headers, cookies, OTPs, refresh tokens, email addresses, phone numbers, and unrelated personal data.
- Capture enough of the request and response to establish the finding, but not entire data sets.
- Keep original evidence securely; use sanitized copies in the report.
- Separate confirmed vulnerabilities from observations, assumptions, and items not tested.

Severity and reporting approach

Severity	General guidance (adapt to demonstrated impact)
Critical	A confirmed issue enabling broad compromise or highly sensitive data access with low prerequisites. Use only when evidence and context justify it.
High	Significant unauthorized access or account compromise with meaningful impact.
Medium	Limited unauthorized data exposure or modification, or a weakness requiring additional conditions.
Low	Limited impact, defense-in-depth gap, or constrained exposure.
Informational	Observation with no demonstrated direct security impact.

Severity must not be assigned from the vulnerability name alone. Consider data sensitivity, number of users potentially affected, required privileges, exploit reliability, and whether the test demonstrated read, write, or delete impact.

Conclusion

This assessment framework covers API reconnaissance, controlled API vulnerability testing, a multi-request crAPI investigation, BOLA verification, authorization matrices, OTP controls, Android static analysis, and integrated pentester decision-making. The final conclusion should be completed only after the actual lab tests are performed.

Final outcome (complete after testing):

Area	Outcome	Evidence reference / notes
API endpoint mapping	[Completed / Partial / Not completed]	[E-01]
Two API weakness tests	[Confirmed / Not reproduced / Inconclusive]	[E-02A, E-02B]
crAPI investigation	[Confirmed / Not reproduced / Inconclusive]	[E-03]
BOLA / authorization	[Confirmed / Not reproduced / Inconclusive]	[E-04, E-05]
OTP review	[Confirmed / Not reproduced / Inconclusive]	[E-06]
JADX APK analysis	[Completed / Partial / Not completed]	[E-07, E-08]

-